WELT 4 · DOPPELSTUNDEN 28 UND 29 (KÜR) · IDE-LEVEL 4.5 UND 4.6

W4·3 – Umformen und Anwendungen

Datei: `AQ_W4_L4_Hafen_Geruest.txt` (Klasse Umformen: Level 4.5 · Expertenpfad, Knoten, MeinStapel: Level 4.6) · Im Lab: **Level 4.5** und **Level 4.6** · Teil 1 in DS 28, Teil 2 in DS 29 (Kür)

Im Zollamt des Hafens wird umgeladen, geprüft und gerechnet: Kisten wandern von der Ladeliste in den Kran und vom Kran in die Warteschlange, die Frachtbriefe enthalten Formeln mit vielen Klammern, und der alte Zollmeister rechnet auf seinem Rechenschieber ganz ohne Klammern. Am Ende des Tages kommt die Flut – und läuft Feld für Feld ins Hafenbecken.

Im Abitur wurden Strukturen ineinander umgeformt (2022 Nachtermin, 2023) und Algorithmen mit Stapel und Schlange verfolgt und erläutert (2025 Nachtermin). Klammerprüfung und Postfix-Rechner sind die klassischen Anwendungen des Stapels; der Flood-Fill aus Welt 3 zeigt, was Schlange und Stapel unterscheidet.

Teil 1 – Umformen, Klammern, Postfix (DS 28)

Infoteil 1: Von einer Struktur in die andere

von → nach	Vorgehen	Reihenfolge danach
Schlange → dynamische Reihung	solange q nicht leer: d.append(q.dequeue())	das vorderste Element steht an Index 0 – Reihenfolge bleibt
dynamische Reihung → Stapel	für i von 0 bis d.getLength() – 1: s.push(d.getItem(i))	Index 0 liegt unten, das letzte Element oben
Stapel → Schlange	solange s nicht leer: q.enqueue(s.pop())	das oberste Element steht vorne
dynamische Reihung → Schlange	für i von 0 bis d.getLength() – 1: q.enqueue(d.getItem(i))	Index 0 steht vorne – Reihenfolge bleibt

- Aus Stapel und Schlange kann man nur durch **Entnehmen** lesen – die Quelle ist danach leer. Ist „unverändert“ gefordert, musst du sie wiederherstellen (AB 2). Aus einer dynamischen Reihung liest `getItem(i)`, **ohne** etwas zu verändern.
- Mit `insertAt(0, x)` statt `append(x)` landet jedes neue Element vorne – die Reihenfolge wird umgedreht.
- Immer angeben, **wo** oben bzw. vorne ist – sonst ist eine Zustandsskizze nicht eindeutig.

Aufgabe 1: Umladen im Hafen

PAPIER ★

Die dynamische Reihung d enthält die Kisten Holz, Erz, Gold, Salz (Index 0 zuerst).

```
1   Stack s = new Stack();
2   for (int i = 0; i < d.getLength(); i++) {
3       s.push(d.getItem(i));
4   }
5   Queue q = new Queue();
6   while (!s.isEmpty()) {
7       q.enqueue(s.pop());
8   }
9   DynArray e = new DynArray();
10  while (!q.isEmpty()) {
11      e.insertAt(0, q.dequeue());
12  }
```

a) Gib den Inhalt von d, s, q und e nach Zeile 4, nach Zeile 8 und nach Zeile 12 an (Stapel oben zuerst, Schlange vorne zuerst, Reihung Index 0 zuerst). **[3 BE]**

nach Zeile	d	s	q	e
4				–
8				–
12				

b) Vergleiche e mit d: Gleiche Reihenfolge? Dieselben Objekte? Begründe. **[1 BE]**

c) In Zeile 11 wird `e.insertAt(0, q.dequeue())` durch `e.append(q.dequeue())` ersetzt. Gib e am Ende an. **[1 BE]**

d) Erläutere, in welchen Zeilen sich die Reihenfolge umdreht, und warum am Ende von a) die ursprüngliche Reihenfolge herauskommt. **[2 BE]**

Infoteil 2: Klammern prüfen mit einem Stapel

In einem richtig geklammerten Ausdruck gehört jede schließende Klammer zur **zuletzt geöffneten**, noch offenen Klammer – genau das liefert ein Stapel (LIFO). Der Algorithmus läuft Zeichen für Zeichen durch den Text:

1. Öffnende Klammer `(`, `[` oder `{` → auf den Stapel legen (push).
2. Schließende Klammer → **Leer?** Ist der Stapel leer, gibt es keine passende öffnende Klammer: Ergebnis falsch. Sonst die oberste Klammer entnehmen (pop) und vergleichen: Passt sie nicht zur Art (z. B. `[` zu `)`), Ergebnis falsch.
3. Alle anderen Zeichen werden ignoriert.
4. Am Ende: Ist der Stapel leer, waren alle Klammern geschlossen → wahr; sonst ist noch eine Klammer offen → falsch.

Der Stapel hat den Inhaltstyp **Zeichen**: `offen.push(c);` legt ein char hinein (automatisch verpackt), `char auf = (Character) offen.pop();` holt es zurück.

Aufgabe 2: Klammern im Frachtbrief

PAPIER ★★

a) Verfolge die Prüfung von `[(k+2)*{m-1}]`: Notiere für jedes Klammerzeichen die Aktion und den Stapel danach, in der letzten Zeile das Ergebnis. **[3 BE]**

Zeichen	Aktion	Stapel (oben → unten)
Ende		

b) Gib das Ergebnis der Prüfung an und begründe mit der Stelle, an der es feststeht. **[4 BE]**

Ausdruck	Ergebnis	Begründung
`([]{)}`		
`((()))`		
`{[}`		
`x)(`		

c) Ein Teammitglied zählt nur: +1 für jede öffnende, −1 für jede schließende Klammer; richtig sei der Ausdruck, wenn am Ende 0 herauskommt. Gib zwei Ausdrücke an, bei denen das Zählverfahren falsch urteilt, und erläutere, warum man einen Stapel braucht. **[2 BE]**

Infoteil 3: Rechnen ohne Klammern – die Postfix-Schreibweise

In der **Postfix-Schreibweise** steht das Rechenzeichen **hinter** seinen beiden Operanden: 3 + 4 wird zu `34+`, (3 + 4) · 2 wird zu `34+2*`, 3 + 4 · 2 wird zu `342*+`. Klammern braucht man nicht mehr – die Reihenfolge legt alles fest. Wir verwenden einstellige Zahlen und die Rechenzeichen + − *, ohne Leerzeichen.

1. Ziffer → ihren Zahlwert auf den Stapel legen. Aus dem Zeichen c wird die Zahl mit `(int) c - (int) '0'` (ASCII-Wert).
2. Rechenzeichen → zwei Werte entnehmen: **zuerst den rechten Operanden b**, dann den linken a; a ∘ b ausrechnen und das Ergebnis wieder auf den Stapel legen.
3. Am Ende liegt genau ein Wert auf dem Stapel: das Ergebnis.

Aufgabe 3: Der Rechenschieber des Zollmeisters

PAPIER ★★

a) Verfolge die Auswertung von `742-*3+` (Stapel vom Inhaltstyp Ganzzahl). **[3 BE]**

Zeichen	Aktion	Stapel (oben → unten)
7		
4		
2		
-		
*		
3		
+		
Ende		

b) Übersetze in Postfix-Schreibweise und berechne mit dem Verfahren: (1) (5 + 3) · (9 − 7) (2) 8 − 2 − 3 (von links nach rechts gerechnet) (3) 8 − (2 − 3). **[3 BE]**

c) Ein Teammitglied entnimmt zuerst den **linken** Operanden. Welches Ergebnis liefert sein Verfahren für `742-*3+`? Warum sind nur manche Ausdrücke betroffen? **[2 BE]**

d) Wie viele Werte liegen beim Auswerten von `2534+*-` höchstens gleichzeitig auf dem Stapel? Gib einen gültigen Ausdruck mit fünf Ziffern an, bei dem es nie mehr als zwei sind. **[1 BE]**

Aufgabe 4: Level 4.5 – Umformen

IDE ★ – ★★★

Arbeite in der Klasse **Umformen** (static), Testklasse `L45UmformenTest`.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	schlangeZuDynArray(q: Schlange): dynamische Reihung (q bleibt unverändert) dynArrayZuStapel(d: dynamische Reihung): Stapel	`basis_`
★★ Erweiterung	klammernOk(s: Zeichenkette): Wahrheitswert	`erweiterung_`
★★★ Experte	postfix(ausdruck: Zeichenkette): Ganzzahl nachGewicht(q: Schlange vom Inhaltstyp Kiste, grenze: Ganzzahl): dynamische Reihung vom Inhaltstyp Kiste (q bleibt unverändert)	`experte_`

- `klammernOk`: Die Tests prüfen auch `""`, `"Hafen"` (keine Klammern → wahr) und Ausdrücke, die mit einer schließenden Klammer beginnen – Leer? vor pop!
- `postfix`: Ein falscher Cast meldet „LAUFZEITFEHLER: falscher Cast“ – liegen auf deinem Stapel wirklich Zahlen und keine Zeichen?

> 🎯 **Im Lab üben**
>
> - **Level 4.5 „Zollamt: Umformen, Klammern, Postfix“** – passt zu Teil 1, Aufgaben 1–4
> - **Trainingsplatz Welt 4:** Zufallsaufgaben zu Stapel, Schlange und dynamischer Reihung (1 Punkt je Aufgabe)
>
> Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

Teil 2 – Kür: Flut, Aufwand und ein eigener Stapel (DS 29)

In Welt 3 habt ihr den Nebel **rekursiv** gelüftet. Jetzt kommt die Flut – und läuft **ohne Rekursion** ins Hafenbecken: Die noch zu bearbeitenden Felder merkt sich eine Schlange oder ein Stapel. Am Ende baut ihr einen Stapel selbst – aus Knoten und Zeigern.

Infoteil 4: Flood-Fill ohne Rekursion

fluteIterativ(karte: Reihung von Reihung von Zeichen, z: Ganzzahl, s: Ganzzahl, mitSchlange: Wahrheitswert): Ganzzahl

(z|s) liegt außerhalb oder karte[z][s] ist nicht '.', 'S' oder 'Z'

wahr	falsch
gib 0 zurück	–

ablage ← erzeuge Schlange (mitSchlange = wahr) bzw. Stapel (falsch)

markiere (z|s) *// besucht ← wahr, merke(z, s); aus '.' wird '+', anzahl zählen*

lege z * 1000 + s in die ablage *// enqueue bzw. push*

solange die ablage nicht leer ist

- pos ← entnimm aus der ablage *// dequeue bzw. pop*
- pz ← pos / 1000; ps ← pos % 1000
- für jeden Nachbarn (nz|ns) von (pz|ps) in der Reihenfolge oben, rechts, unten, links
  - (nz|ns) im Feld und begehbar und noch nicht besucht

wahr	falsch
markiere (nz|ns) *// BEIM Einfügen markieren*	–
lege nz * 1000 + ns in die ablage	

gib anzahl zurück *// Felder, die von '.' zu '+' wurden*

Abbildung 1: fluteIterativ (Level 4.6) – „ablage“ ist eine Schlange oder ein Stapel

- Begehbar sind '.', 'S' und 'Z'. Markieren heißt: als besucht merken; aus '.' wird '+', S und Z bleiben stehen.
- Eine Position (z|s) wird als **eine Ganzzahl** gespeichert: pos = z · 1000 + s, zurück mit z = pos / 1000 und s = pos % 1000.
- **Markiert wird beim Einfügen**, nicht beim Entnehmen – so kommt kein Feld zweimal in die Ablage.
- Nachbarn immer in der Reihenfolge **oben, rechts, unten, links**.

Aufgabe 5: Die Flut auf der Lichtung

PAPIER ★★

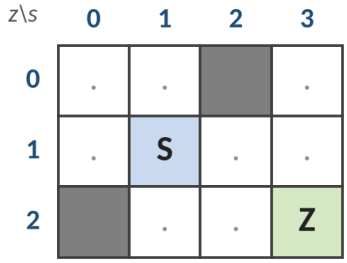


Abbildung 2: Die Lichtung (keine Randmauer)

Auf der Lichtung aus Welt 3 wird `fluteIterativ(karte, 1, 1, …)` aufgerufen. Positionen schreibst du als (z|s). In jeder Zeile der Tabellen steht eine Entnahme: das entnommene Feld, die dabei neu markierten Nachbarn (in der Reihenfolge oben, rechts, unten, links) und der Inhalt der Ablage danach.

a) Führe den Algorithmus **mit einer Schlange** aus. **[4 BE]**

Nr.	entnommen	neu markiert	Schlange (vorne → hinten)
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Zeile 0: Start (1|1) wird markiert und eingereiht – entnommen: –, neu markiert: (1|1), Schlange: (1|1).

b) Führe den Algorithmus **mit einem Stapel** aus. **[4 BE]**

Nr.	entnommen	neu markiert	Stapel (oben → unten)
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

c) Vergleiche: Welche Rückgabe liefern beide Aufrufe, und wie sieht die Karte am Ende aus? Was unterscheidet sich? Welche Variante breitet sich „in Wellen“ um S aus? **[2 BE]**

d) Die rekursive Methode lueften aus Welt 3 markiert die Lichtung in der Reihenfolge (1|1) (0|1) (0|0) (1|0) (1|2) (1|3) (0|3) (2|3) (2|2) (2|1). Erläutere, warum die Stapel-Variante – obwohl auch sie „erst in die Tiefe“ geht – eine andere Reihenfolge liefert. **[2 BE]**

e) Begründe, warum die Position als z · 1000 + s gespeichert wird und nicht als z · 10 + s. **[1 BE]**

Aufgabe 6: Eine Frage des Aufwands

PAPIER ★★★

Ein Mitschüler baut die Schlange mit einer dynamischen Reihung nach: enqueue = `append(x)`, dequeue = `getItem(0)` und danach `delete(0)`. Wir nehmen an, dass die dynamische Reihung intern eine Reihung benutzt: `delete(i)` und `insertAt(i, x)` verschieben **jedes** Element hinter Position i um eine Stelle – jede Verschiebung ist ein Schritt. Eine Schlange der Bibliothek kommt pro Operation mit einer festen Zahl von Schritten aus (keine Verschiebungen).

a) n Kisten werden mit append eingereiht und danach alle mit delete(0) entnommen. Bestimme die Anzahl der Verschiebungen für n = 5 und leite eine Formel für n her. Ergänze die Tabelle. **[3 BE]**

n	5	10	100	1000
Verschiebungen beim Entnehmen				

b) Wie viele Verschiebungen kostet es, n Kisten nacheinander mit `insertAt(0, k)` vorne in eine leere dynamische Reihung einzufügen? **[1 BE]**

c) Beurteile: Für welche Anwendungen ist trotzdem die dynamische Reihung die bessere Wahl, für welche die Schlange? **[2 BE]**

Infoteil 5: Expertenpfad – ein Stapel aus Knoten (Vertiefung, im Abitur nicht verlangt)

Wie ist Stack innen gebaut? Eine Möglichkeit ist eine **Kette aus Knoten**: Jeder Knoten speichert einen Inhalt und einen Verweis auf seinen **Nachfolger** (den Knoten darunter). Der Stapel selbst merkt sich nur den obersten Knoten `kopf`; `null` bedeutet „leer“. Es gibt keine feste Größe – der Stapel ist wirklich **dynamisch**.

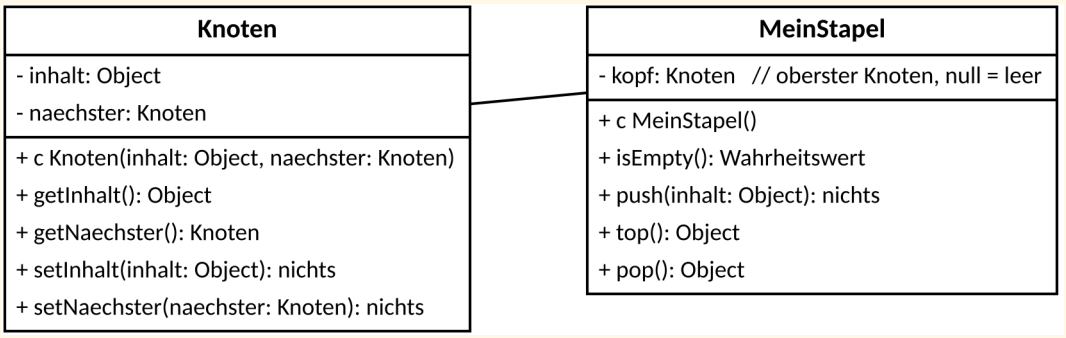


Abbildung 3: Klassenkarten Knoten und MeinStapel (Level 4.6, Experte)

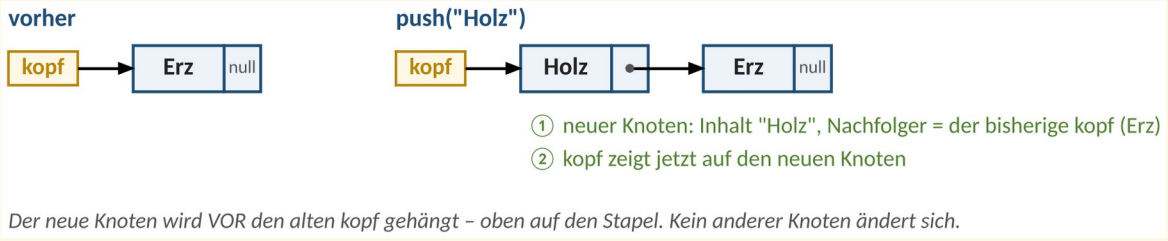


Abbildung 4: push als Zeigerbild

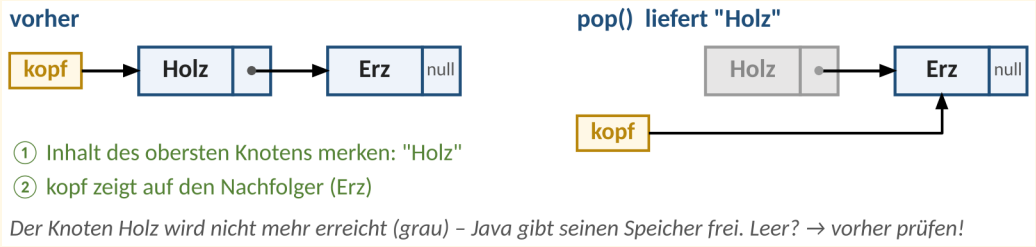


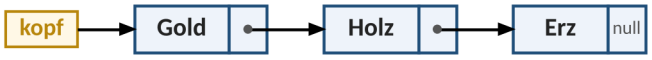
Abbildung 5: pop als Zeigerbild

MeinStapel verhält sich wie Stack: `top()` und `pop()` auf einem leeren Stapel lösen einen Laufzeitfehler aus – in deinem Code mit `throw new IllegalStateException("Der Stapel ist leer.");`.

Aufgabe 7: Zeigerbilder

PAPIER ★★★

Ein MeinStapel-Objekt sieht so aus:



a) Zeichne das Zeigerbild nach `push("Salz")`. **[1 BE]**

b) Danach wird zweimal `pop()` aufgerufen. Gib die Rückgaben an und zeichne das Zeigerbild. Welche Knoten sind nicht mehr erreichbar? **[2 BE]**

c) Ein Teammitglied schreibt in push `kopf = new Knoten(inhalt, null);`. Zeichne das Bild nach `push("Salz")` auf den Stapel aus der Abbildung oben und erläutere den Fehler. **[2 BE]**

d) Begründe, warum top() und pop() eine Leer-Abfrage brauchen, push aber nicht. **[1 BE]**

Aufgabe 8: Level 4.6 – Expertenpfad

IDE ★ – ★★★

Klasse **Expertenpfad** (static) sowie die Klassen **Knoten** und **MeinStapel**, die du komplett selbst schreibst. Testklasse `L46ExpertenpfadTest`.

Stufe	Aufgabe	Tests beginnen mit
★ Basis	fluteIterativ(karte: Reihung von Reihung von Zeichen, z: Ganzzahl, s: Ganzzahl, mitSchlange: Wahrheitswert): Ganzzahl – Endkarte und Anzahl mit Schlange	`basis_`
★★ Erweiterung	dieselbe Methode: mit Stapel, Karte ohne Randmauer, genaue Reihenfolge in spur	`erweiterung_`
★★★ Experte	Klassen Knoten und MeinStapel nach den Klassenkarten (Abbildung 3), dann den Block LEVEL-4.6 freischalten	`experte_`

- Rufe `merke(z, s)` **direkt beim Markieren** jedes Feldes auf (auch für S und Z). Deine Tabellen aus Aufgabe 5 sind genau die spur, die die Tests für die Lichtung erwarten.
- **Freischalten:** In `L46ExpertenpfadTest` und im Hauptprogramm je die Zeile mit `/* LEVEL-4.6-START` und die Zeile mit `LEVEL-4.6-ENDE */` löschen – erst, wenn Knoten und MeinStapel kompilieren.
- Die Tests vergleichen MeinStapel in 400 zufälligen Befehlen mit Stack (immer dieselbe Folge) und prüfen die IllegalStateException bei leerem Stapel.

Für Schnelle: Schlangen bauen und Wellen zählen

- **Schlange aus zwei Stapeln:** enqueue legt auf einen Stapel `ein`; dequeue entnimmt von einem Stapel `aus` – ist `aus` leer, wird vorher **alles** von `ein` nach `aus` umgeschichtet. Spiele enqueue A, B, C · dequeue · enqueue D · dequeue · dequeue · dequeue durch. Kommen die Elemente in der richtigen Reihenfolge? Wie oft wird insgesamt umgeschichtet?
- **Wellen zählen:** Erweitere die Schlangen-Variante so, dass sie für jedes Feld den Abstand zu S (Anzahl der Schritte) speichert. Warum ist das die Länge eines **kürzesten** Weges? Vergleiche mit dem Backtracking aus Welt 3.
- **MeinSchlange:** Baue eine Schlange aus Knoten mit zwei Verweisen `kopf` (vorne) und `ende` (hinten). Welche Sonderfälle gibt es beim ersten enqueue und beim letzten dequeue?

🎯 Im Lab üben

- **Level 4.6 „Kür: Die Flut – Schlange gegen Stapel“** – passt zu Teil 2, Aufgaben 5–7

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

Abi-Check Umformen und Anwendungen

- ☐ Beim Umformen die entstehende Reihenfolge angeben: Jeder Weg durch einen Stapel dreht um, Schlange und append erhalten die Reihenfolge.
- ☐ Quelle wiederherstellen, wenn sie unverändert bleiben soll; Zustände mit oben/vorne bzw. Index 0 eindeutig skizzieren.
- ☐ Klammerprüfung: Leer? vor jedem pop, Art der Klammer vergleichen, am Ende `isEmpty()` zurückgeben.
- ☐ Postfix: Der **zuerst** entnommene Wert ist der **rechte** Operand; Ziffer → Zahl mit `(int) c - (int) '0'`.
- ☐ Casts beim Entnehmen: `(Character)`, `(Integer)`, `(Kiste)`; Eignung und Aufwand mit den benötigten Operationen begründen.